

이 자료 사용 방법

1. 수업·강의·자습에 그대로 쓰셔도 됩니다. 출처만 남겨 주세요.
2. PDF 는 화면으로 읽는 용도, PPTX 는 발표용입니다. PPTX 는 Pretendard 글꼴이 설치된 컴퓨터에서 가장 똑같이 보입니다.
3. 출처 표기 예: 이애본(드림아이티비즈), 「자료 제목」, 홍문원, useful-ai.dreamitbiz.com, CC BY-NC-ND 4.0
4. 자료 안의 프롬프트는 복사해 바로 쓸 수 있습니다. [대괄호]만 내 상황으로 바꾸세요.
5. 모든 자료와 영상 강의는 useful-ai.dreamitbiz.com 에서 다시 받을 수 있습니다.

저작권 · 이용 조건 (CC BY-NC-ND 4.0)

- 저작권 © 2026 이애본(드림아이티비즈 DreamIT Biz). 발행 홍문원(洪文院, 출판사 신고번호 제2026-000026호). 이애본이 2026년 대학·기관 강의 현장에서 쓴 강의안을 전자출판용으로 다시 묶은 것입니다. ISBN: 등록 후 기재.
- 이 자료는 크리에이티브 커먼즈 저작자표시-비영리-변경금지 4.0 국제(CC BY-NC-ND 4.0) 라이선스를 따릅니다. creativecommons.org/licenses/by-nc-nd/4.0/deed.ko
- 저작자 표시(BY): 출처를 밝히면 그대로 복사해 수업·강의·자습에 쓸 수 있습니다. 비영리(NC): 유료 강의·판매·광고 수익 목적에는 쓸 수 없습니다. 변경금지(ND): 내용을 고치거나 잘라 재조합해 배포할 수 없습니다.
- 자료 안의 외부 서비스 화면 캡처와 상표는 교육 목적 인용이며 각 서비스의 소유입니다. AI 생성 이미지는 생성 도구의 이용 약관을 따릅니다.
- 라이선스 범위 밖의 이용(영리·변형·2차 저작)은 먼저 문의해 주세요. aebon@dreamitbiz.com · 카카오톡 ID aebon

A07 · 바이트코딩

클로드 코드와 제미나이로 바이트코딩 포트폴리오 배포

터미널 AI 에이전트와 함께 만들고 GitHub Pages 로 공개하기

이애본 지음 · 홍문원(洪文院) 발행

CC BY-NC-ND 4.0 · useful-ai.dreamitbiz.com

이 강의안의 순서

1	바이브코딩과 AI 코딩 도구	이해	5	기능 더하기	제작
2	개발 환경 준비	준비	6	다듬기 — 성능·접근성·보안	품질
3	AI 에게 잘 요청하기	준비	7	GitHub Pages 배포	배포
4	포트폴리오 뼈대 만들기	제작	8	문제 해결과 최종 점검	배포

이 강의안을 마치면

목표 ①	Claude Code·Gemini CLI 가 하는 일을 설명한다
목표 ②	역할·목표·제약·맥락을 갖춘 요청문을 쓴다
목표 ③	반응형 포트폴리오를 AI 와 함께 만든다
목표 ④	AI 가 만든 코드를 직접 확인하고 고친다
목표 ⑤	GitHub Pages 로 내 사이트 주소를 만든다

결과물 — 인터넷에서 열리는 내 포트폴리오 주소 하나

시작 전에 갖출 것

프로그램

01

VS Code · Git · Node.js(LTS)
Chrome 같은 최신 브라우저

계정

02

GitHub 계정(배포용)
Claude 또는 Google 계정(AI 도구용)

기초 지식

03

HTML 태그·CSS 선택자·JS 문법을
조금 알면 AI 결과를 읽기 쉽다

마음가짐

04

AI 가 쓴 코드도 내 이름으로 나간다
— 읽고 확인한 뒤에 쓴다

PART 1

바이트코딩과 AI 코딩 도구

말로 의도를 전하고, 결과를 보고, 다시 고친다 — 새 개발 방식과 그 도구를 먼저 이해합니다

개념

바이트코딩이란

01

의도를 말한다

무엇을 만들고 싶은지
자연어로 설명한다

02

결과를 본다

AI 가 쓴 코드를
브라우저에서 바로 확인한다

03

다시 고친다

마음에 안 드는 곳을
대화로 하나씩 바로잡는다

비유 — 내가 감독, AI 는 손이 빠른 조수입니다. 조수가 빨라도 무엇을 찍을지는 감독이 정합니다

비교

예전 방식과 바이트코딩

직접 타이핑

문법과 세부를 내가 모두 쓴다

- 순서대로 한 줄씩 작성
- 문법 오류가 큰 장벽
- 속도 = 타이핑 속도

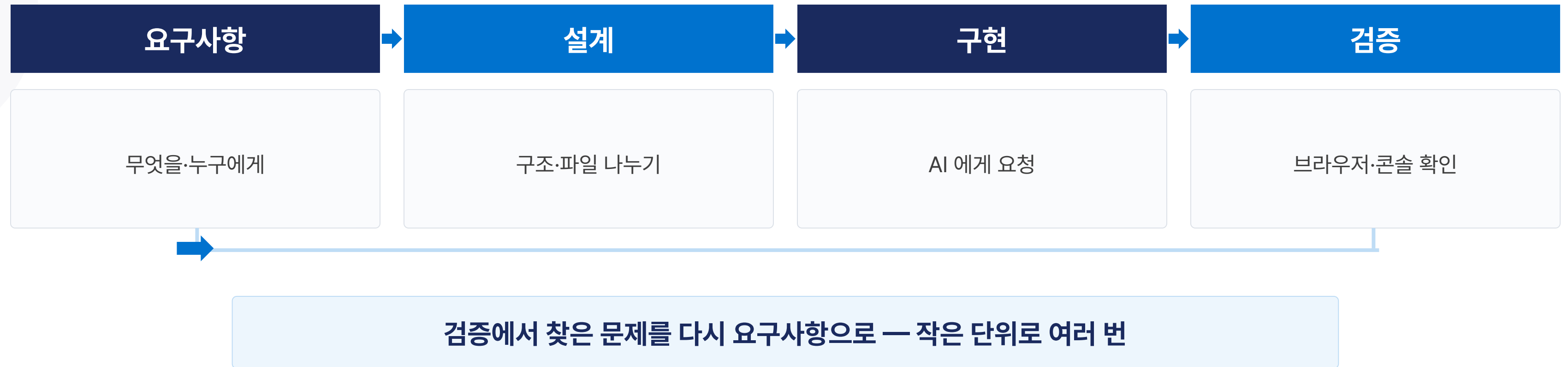
VS

대화로 개발

의도와 설계를 내가 정한다

- 요청 → 결과 → 수정의 반복
- 설계·검토가 핵심 역할
- 속도 = 생각을 정리하는 속도

바이트코딩의 반복 고리



한 바퀴를 작게 돌수록 실수가 작을 때 잡힙니다

채팅형 AI 와 터미널 에이전트

채팅형(웹.앱)

claude.ai · gemini.google.com

- 대화창에 코드를 붙여 넣어 질문
- 답을 내가 복사해 파일에 붙임
- 내 폴더는 보지 못함

터미널 에이전트

Claude Code · Gemini CLI

VS

- 프로젝트 폴더 안에서 실행
- 파일을 직접 읽고 고침
- 명령 실행 전 내 승인을 받음

Claude Code

무엇인가

01

Anthropic 이 만든
터미널용 코딩 에이전트

어떻게 쓰나

02

프로젝트 폴더에서
claude 를 실행해 대화

규칙 파일

03

CLAUDE.md 에 적은
프로젝트 규칙을 읽는다

이용하려면 Claude 유료 요금제나 API 계정이 필요합니다. 요금제·조건은 바뀔 수 있으니 공식 안내를 확인하세요

Gemini CLI

무엇인가

01

Google 이 공개한
오픈소스 터미널 에이전트

어떻게 쓰나

02

프로젝트 폴더에서
gemini 를 실행해 대화

규칙 파일

03

GEMINI.md 에 적은
프로젝트 규칙을 읽는다

Google 계정으로 로그인해 쓸 수 있습니다. 무료 사용 한도와 조건은 바뀔 수 있으니 공식 안내를 확인하세요

비교

두 도구를 나란히 보면

항목	Claude Code	Gemini CLI
만든 곳	Anthropic	Google(오픈소스)
실행	터미널에서 claude	터미널에서 gemini
로그인	Claude 계정 또는 API 키	Google 계정 또는 API 키
규칙 파일	CLAUDE.md	GEMINI.md
공통점	파일 읽기·수정, 명령 실행 — 실행 전 승인	파일 읽기·수정, 명령 실행 — 실행 전 승인

둘 중 하나만 있어도 이 강의안 전체를 따라올 수 있습니다

모델 성능·컨텍스트 길이는 버전마다 달라 이 표에 실지 않습니다

AI 와 코딩할 때 지킬 다섯 가지

① 비밀번호·API 키·개인정보를 대화에 붙여 넣지 않는다

② 명령 실행 승인 전에 무슨 명령인지 읽는다

③ 작은 단위로 요청하고 바로 확인한다

④ AI 가 쓴 코드도 직접 읽고 이해한 뒤 쓴다

⑤ 자주 커밋해 언제든지 되돌릴 수 있게 한다

PART 2

개발 환경 준비

편집기·버전 관리·실행 환경·AI 에이전트 — 네 가지를 차례로 설치하고 확인합니다

네 가지를 차례로 설치합니다

각 단계가 끝나면 터미널에서 버전이 찍히는지 확인하고 다음으로 넘어갑니다

VS Code

code.visualstudio.com
코드 편집기

Git

git-scm.com
macOS 는 Homebrew 도
가능

Node.js LTS

nodejs.org
npm 이 함께 설치됨

AI 에이전트

Claude Code 또는
Gemini CLI

함께 깔아 두면 좋은 확장

Live Server

01

저장할 때마다
브라우저가 새로고침

Prettier

02

코드 들여쓰기·모양을
자동으로 정리

한국어 팩

03

메뉴를 한국어로
(선택 사항)

확장은 왼쪽 확장 탭(Windows Ctrl+Shift+X · macOS Cmd+Shift+X)에서 이름으로 검색해 설치합니다

터미널에서 버전 확인

```
1 git --version
2 node -v
3 npm -v
4 code --version # VS Code 를 터미널에서 여는 명령
```

숫자는 사람마다 다릅니다 — 버전이 찍히면 설치 성공, 「명령을 찾을 수 없음」이면 재설치

내 이름과 이메일 등록 (한 번만)

```
1 git config --global user.name "[내 이름]"
2 git config --global user.email "[GitHub 이메일]"
3 git config --global init.defaultBranch main
4
5 git config --list # 등록된 값 확인
```

이메일은 GitHub 계정과 같은 것을 권장합니다 — 커밋이 내 계정에 연결됩니다

Claude Code 설치와 첫 실행

- 1 공식 문서의 설치 명령 실행**
설치 방법은 바뀔 수 있으니 Anthropic 공식 문서를 따른다
- 2 프로젝트 폴더로 이동**
cd [폴더 이름]
- 3 claude 입력**
처음 실행하면 브라우저에서 로그인 절차가 열린다
- 4 /init 으로 규칙 파일 만들기**
폴더를 살펴보고 CLAUDE.md 초안을 써 준다

확인

**claude 가 대화 입력을
기다리면 준비 끝**

Gemini CLI 설치와 첫 실행

1 공식 문서의 설치 명령 실행
npm 으로 설치 — Node.js 가 먼저 있어야 한다

2 프로젝트 폴더로 이동
cd [폴더 이름]

3 gemini 입력
처음 실행하면 Google 계정 로그인 방식을 고른다

4 GEMINI.md 만들기
프로젝트 규칙을 적어 두면 매번 읽는다

확인

**gemini 가 대화 입력을
기다리면 준비 끝**

폴더를 만들고 에이전트 열기

```
1 mkdir my-portfolio      # 폴더 만들기
2 cd my-portfolio         # 폴더 안으로 이동
3 git init                # Git 저장소로 시작
4 code .                  # 이 폴더를 VS Code 로 열기
5
6 claude                  # Claude Code 시작 (또는)
7 gemini                  # Gemini CLI 시작
```

폴더 이름은 영어 소문자와 하이픈(-)으로 — 나중에 사이트 주소의 일부가 됩니다

요청 한 번에 일어나는 일

에이전트는 파일을 바꾸거나 명령을 실행하기 전에 내 승인을 묻습니다

요청	살펴보기	승인	확인
내가 원하는 것을 말로 적는다	폴더의 파일을 읽고 계획을 세운다	바꿀 내용·실행할 명령을 보여 주고 묻는다	브라우저에서 결과를 보고 다음 요청

API 키는 언제 필요한가

이 강의안

01

정적 포트폴리오에는
필요 없다 — 로그인으로 충분

필요한 경우

02

내 코드가 AI API 를
직접 호출할 때만

그때 주의

03

.env 는 .gitignore 에 넣고
요금 한도를 먼저 정한다

브라우저로 보내는 JS 에 넣은 키는 누구나 볼 수 있습니다. 키는 서버나 서버리스 함수 쪽에만 둡니다

환경 준비 완료 확인

● 점검

- ① VS Code 가 열리고 확장(Live Server 등)이 켜져 있다
- ② `git --version` · `node -v` 가 버전을 출력한다
- ③ `git config` 에 이름·이메일이 등록돼 있다
- ④ 프로젝트 폴더에서 `claude` 또는 `gemini` 가 실행된다
- ⑤ `index.html` 을 Live Server 로 열면 `127.0.0.1` 주소가 뜬다

다섯 줄을 모두 통과하면 PART 3 으로

PART 3

AI 에게 잘 요청하기

같은 도구라도 요청이 분명하면 고칠 일이 줄어듭니다 — 요청문의 뼈대를 익힙니다

명확한 요청의 세 요소

역할

01

어떤 전문가처럼
답할지 정해 준다

목표

02

무엇을 얻고 싶은지
결과물을 분명히

제약

03

쓸 기술·파일·금지할 것
경계를 적는다

예 — 「외부 라이브러리 없이 HTML·CSS·순수 JS 로, 모바일 우선 반응형으로 만들어 줘」

AI가 알아야 할 세 가지 맥락

파일 구조

01

어떤 파일이 어디 있고
무엇을 새로 만들지

스타일 가이드

02

색·글꼴·여백 규칙을
구체적인 값으로

참조 코드

03

이미 쓴 코드 패턴을
따르라고 알려 준다

에이전트는 폴더를 직접 읽지만, 규칙(색·글꼴·금지 사항)은 말해 주지 않으면 모릅니다

프로젝트 규칙 파일 만들기

구조 — 역할 + 목표 + 제약 + 출력 형식

“이 폴더는 내 개인 포트폴리오 사이트야. 프로젝트 규칙 파일([CLAUDE.md 또는 GEMINI.md])을 만들어 줘.
내용: ① HTML·CSS·순수 JS 만 사용 ② 파일은 index.html·css/style.css·js/main.js ③ 메인 색 [#
색상코드], 글꼴 [글꼴 이름] ④ 모바일 우선 반응형 ⑤ 파일 이름은 영어 소문자와 하이픈. 만들기 전에 초안을
먼저 보여 줘.”

산출물

에이전트가 매번 읽는 프로젝트 규칙 파일

한 번에 다 시키지 않는다

큰 일을 세 단계로 쪼개고, 단계마다 브라우저에서 확인한 뒤 다음 단계로 갑니다

1 구조

HTML 골격과
섹션 순서

2 스타일

CSS 색·배치·
반응형

3 기능

JS 인터랙션·
접근성·최적화

리뷰를 요청할 때

구조 — 검토 범위 + 확인 항목 + 출력 형식

“index.html 을 검토해 줘. 특히 ① 시맨틱 태그를 알맞게 썼는지 ② 이미지 alt 가 빠진 곳 ③ 필요 없이 겹친 div 를 확인해 줘. 고칠 곳은 바꾸기 전·후 코드와 이유를 함께 보여 주고, 내가 승인하면 그때 파일을 고쳐 줘.

산출물

이유가 붙은 수정 제안 목록

오류를 물어볼 때

구조 — 에러 전문 + 상황 + 재현 순서 + 요청

[에러 로그] Uncaught TypeError: Cannot read properties of null (reading 'classList')

[상황] js/main.js 의 [함수 이름]에서 페이지를 열자마자 납니다.

[재현] 새로그침 → 메뉴 버튼 클릭

[요청] 원인을 먼저 설명하고, DOM 이 준비되는 시점을 고려한 수정안을 보여 줘.

핵심

「안 돼요」 대신 에러 전문·상황·재현 순서를 함께

PART 4

포트폴리오 뼈대 만들기

폴더 구조 → HTML 골격 → CSS 변수 → 반응형 격자 순서로 쌓습니다

표준 폴더 구조

경로	역할
index.html	첫 화면(메인 페이지)
css/style.css	모든 스타일
js/main.js	메뉴·다크모드 같은 동작
images/	사진·아이콘(파일명은 소문자)
README.md	프로젝트 설명
.gitignore	Git 에 올리지 않을 파일 목록

구조가 표준에 가까울수록 사람도 AI 도 파일을 빨리 찾습니다

HTML 골격 요청

구조 — 목표 + 필수 요소 + 확인 요청

“index.html 에 포트폴리오 골격을 만들어 줘. ① lang="ko" 와 viewport 메타 ② header·nav·main·footer 시맨틱 태그 ③ main 안에 hero·about·projects·contact 섹션 ④ 키보드 사용자를 위한 '본문 바로가기' 링크 ⑤ CSS·JS 파일 연결. 내용은 [이름]·[한 줄 소개] 자리만 비워 뒀.”

확인

Live Server 로 열어 섹션 순서가 맞는지 본다

index.html 골격 (요약)

```
1 <!doctype html>
2 <html lang="ko">
3 <head>
4   <meta charset="utf-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1">
6   <title>[이름] 포트폴리오</title>
7   <link rel="stylesheet" href="css/style.css">
8   <script src="js/main.js" defer></script>
9 </head>
10 <body>
11   <a class="skip-link" href="#main">본문 바로가기</a>
12   <header><nav aria-label="주 메뉴">...</nav></header>
13   <main id="main">
14     <section id="about" aria-labelledby="about-title">...</section>
15     <section id="projects" aria-labelledby="projects-title">...</section>
16   </main>
17   <footer>© [연도] [이름]</footer>
18 </body>
19 </html>
```

script 에 defer 를 붙이면 HTML 을 다 읽은 뒤 JS 가 실행됩니다

CSS 변수로 스타일 시작

구조 — 스타일 가이드(구체 값) + 규칙

“css/style.css 를 만들어 줘. :root 에 색·글꼴·여백 변수를 먼저 정의하고 모든 규칙이 이 변수를 쓰게 해 줘. 메인 색 [#색상코드], 글자색 [#색상코드], 배경색 [#색상코드], 글꼴 [글꼴 이름]. 글자와 배경의 명도 대비는 4.5:1 이상으로 맞춰 줘.

장점

색을 바꿀 때 한 곳만 고치면 된다 — 다크모드의 바탕

style.css — 변수 정의

```
1  :root {
2    --color-primary: #1d4ed8; /* 예시 값 */
3    --color-text: #1f2937;
4    --color-muted: #6b7280;
5    --color-bg: #ffffff;
6    --font-main: 'Pretendard', sans-serif;
7    --space: 1rem;
8  }
9
10 body {
11   margin: 0;
12   font-family: var(--font-main);
13   color: var(--color-text);
14   background: var(--color-bg);
15   line-height: 1.6;
16 }
```

색상 코드는 예시입니다 — 내 사이트의 색으로 바꿔 쓰세요

반응형을 요청하는 세 가지 원칙

01

모바일 우선

작은 화면을 기본으로 쓰고
넓어질 때 덧붙인다

02

유동 단위

고정 px 대신
%·rem·vw 를 쓴다

03

분기점 명시

몇 px 에서 무엇이
바뀌는지 적어 준다

viewport 메타 태그가 없으면 휴대폰에서 미디어 쿼리가 기대대로 동작하지 않습니다

프로젝트 카드 격자

구조 — 목표 + 반응형 조건 + 접근성 조건

“projects 섹션에 프로젝트 카드 격자를 만들어 줘. ① 카드는 article 태그, 썸네일·제목·설명·사용 기술·데모/코드 링크 ② 넓은 화면 3열, 좁은 화면 1열 — 미디어 쿼리 없이 auto-fit 으로 ③ 마우스를 올리면 살짝 떠오르고(transform) 키보드 포커스에도 같은 표시 ④ 아이콘 링크에는 aria-label.

확인

창 폭을 줄여 3열 → 1열로 바뀌는지 본다

style.css — 카드 격자

```
1  .projects {
2    display: grid;
3    grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
4    gap: 1.5rem;
5  }
6
7  .project-card {
8    border-radius: 12px;
9    box-shadow: 0 2px 8px rgb(0 0 0 / 0.1);
10   transition: transform 0.3s ease;
11 }
12
13 .project-card:hover,
14 .project-card:focus-within {
15   transform: translateY(-6px);
16 }
```

transform 은 레이아웃을 다시 계산하지 않아 움직임이 부드럽습니다

PART 5

기능 더하기

메뉴·소개·연락 폼·다크모드·스크롤 효과·필터 — 하나씩 요청하고 하나씩 확인합니다

머리글과 메뉴

구조 — 배치 + 접근성 조건

“header 를 만들어 줘. ① 왼쪽 로고, 오른쪽 메뉴 ② 스크롤해도 위에 붙어 있게(position: sticky) ③ 「본문 바로가기」 링크는 평소 숨기고 Tab 으로 포커스될 때만 보이게 ④ 좁은 화면에서는 메뉴 버튼으로 열고 닫기, 버튼에 aria-expanded.

확인

마우스 없이 Tab 키만으로 메뉴를 열고 이동해 본다

소개(About) 섹션

구조 — 들어갈 내용 + 배치 + 접근성

“about 섹션을 채워 줘. ① 프로필 사진(alt 는 「[이름] 프로필 사진」) ② 자기소개 [세 문장] ③ 기술 목록 [HTML, CSS, JavaScript ...] ④ 「연락하기」 버튼 ⑤ 넓은 화면은 사진·글 좌우 배치, 좁은 화면은 위아래로 쌓기. 경력 타임라인은 [연도 — 내용] 형식의 목록으로 넣어 줘.

주의

자기소개 글은 AI 초안을 내 말로 다시 쓴다

Formspree 로 폼 받기

GitHub Pages 는 서버가 없어 폼 데이터를 받지 못합니다 — 받아 주는 외부 서비스를 씁니다

가입·폼 생성

formspree.io 에서
새 폼 만들기

주소 받기

formspree.io/f/[폼 ID]
형태의 주소

HTML 연결

form 의 action 에
주소, method=POST

테스트

직접 보내 보고
메일 수신 확인

연락 폼 연결

구조 — 서비스 + 검증 + 사용자 안내

“contact 섹션에 연락 폼을 만들어 줘. 정적 사이트라 Formspree 같은 폼 서비스로 보낼 거야. action 은 [폼 서비스 주소]. ① 이름·이메일·메시지 필드, 각각 label 연결 ② required·type="email" 로 브라우저 기본 검증 ③ 스팸 방지용 숨김 필드 ④ 보낸 뒤 성공·실패 메시지를 화면에 표시.

확인

내 메일로 테스트 메시지가 도착하는지 본다

다크모드 토글

구조 — 동작 + 저장 + 기본값

“다크모드 토글 버튼을 만들어 줘. ① :root 색 변수를 [data-theme="dark"] 에서 다시 정의 ② 버튼을 누르면 html 의 data-theme 을 바꾸고 localStorage 에 저장 ③ 처음 온 사람은 운영체제 설정(prefers-color-scheme)을 따르기 ④ 버튼에 aria-pressed 로 현재 상태 표시.

확인

새로고침해도 고른 테마가 유지되는지 본다

main.js — 다크모드 토글

```
1  const root = document.documentElement;
2  const btn = document.querySelector('#theme-toggle');
3
4  const saved = localStorage.getItem('theme');
5  const prefersDark = matchMedia('(prefers-color-scheme: dark)').matches;
6  setTheme(saved ?? (prefersDark ? 'dark' : 'light'));
7
8  btn.addEventListener('click', () => {
9    const next = root.dataset.theme === 'dark' ? 'light' : 'dark';
10    setTheme(next);
11    localStorage.setItem('theme', next);
12  });
13
14  function setTheme(theme) {
15    root.dataset.theme = theme;
16    btn.setAttribute('aria-pressed', theme === 'dark');
17  }
```

저장값이 없으면 운영체제 설정을 따릅니다

스크롤하면 나타나는 효과

구조 — 효과 + 구현 방식 + 배려

“섹션이 화면에 20%쯤 들어오면 아래에서 부드럽게 떠오르게 해 줘. ① scroll 이벤트 대신 IntersectionObserver 사용 ② 보이면 visible 클래스를 붙이고 한 번 나타난 요소는 관찰 해제(unobserve) ③ 움직임 줄이기 설정(prefers-reduced-motion)인 사람에게는 효과 없이 바로 보이게.

확인

스크롤할 때 각 섹션이 한 번씩만 나타나는지 본다

프로젝트 필터

구조 — 데이터 규칙 + 동작 + 접근성

“프로젝트 카드 위에 태그 필터 버튼을 만들어 줘. ① 각 카드에 `data-tags="[태그1 태그2]"` ② 버튼을 누르면 `aria-pressed` 를 바꾸고, 선택한 태그를 모두 가진 카드만 보이게(every 사용) ③ 숨길 때는 `hidden` 클래스를 토글 ④ 「전체」 버튼으로 초기화.

확인

태그 두 개를 고르면 둘 다 가진 카드만 남는지 본다

모달 창을 요청할 때 넣을 네 조건

열릴 때

01

배경 스크롤 잠금
포커스는 모달 안 첫 요소로

Tab 이동

02

Tab·Shift+Tab 이 모달 안에서만
돌게(포커스 트랩)

닫기

03

ESC·닫기 버튼·배경 클릭
어느 것으로도 닫힌다

닫힌 뒤

04

포커스를 처음 누른
카드로 되돌린다

포트폴리오 기본 완성 확인

● 점검

- ① 개발자 도구 기기 모드 360·390px 에서 가로 스크롤이 없다
- ② 768px(태블릿)·1280px(노트북)에서 배치가 자연스럽다
- ③ Tab 키만으로 모든 링크·버튼에 갈 수 있고 포커스가 보인다
- ④ 콘솔(F12)에 빨간 오류가 없다
- ⑤ 다크모드·필터·폼이 모두 동작한다

여기까지 됐으면 커밋하고 PART 6 으로

PART 6

다듬기 — 성능·접근성·보안

빨리 뜨고, 누구나 쓸 수 있고, 안전한 사이트 — 측정하고 고치는 법을 익힙니다

사용자 경험을 재는 세 지표

LCP

01

가장 큰 내용이 뜨는 시간
좋은 2.5초 이하

INP

02

클릭·입력에 반응하는 시간
좋은 200ms 이하

CLS

03

화면이 덜컥 밀리는 정도
좋은 0.1 이하

INP 는 2024년 3월부터 FID 를 대신합니다. Lighthouse(Chrome 개발자 도구)·PageSpeed Insights 로 배포 주소를 잡니다

이미지 최적화 네 가지

새 형식

01

AVIF·WebP 를 먼저,
JPG 는 대비용으로

크기 명시

02

width·height 를 적어
자리를 미리 잡는다(CLS)

늦게 불러오기

03

화면 아래 이미지만
loading="lazy"

첫 화면 이미지

04

lazy 대신
fetchpriority="high"(LCP)

picture 태그로 형식 고르기

```
1 <picture>
2   <source srcset="images/hero.avif" type="image/avif">
3   <source srcset="images/hero.webp" type="image/webp">
4   
8 </picture>
```

브라우저는 위에서부터 읽을 수 있는 첫 형식을 고릅니다

적은 노력으로 큰 효과 — 접근성 점검 여섯 가지

대체 텍스트

01

뜻 있는 이미지는 alt,
장식 이미지는 alt=""

폼 레이블

02

모든 입력칸에
label 을 연결

명도 대비

03

본문 글자 4.5:1 이상
(WCAG AA)

포커스 순서

04

Tab 순서 = 보이는 순서
tabindex 남용 금지

자동 검사

05

Lighthouse·axe 로
먼저 훑는다

사람 검사

06

자동 검사가 못 잡는 것은
키보드로 직접 써 본다

보안 헤더는 호스팅에 따라 다르다

01 GitHub Pages

응답 헤더를
직접 설정할 수 없다

02 Netlify·Cloudflare Pages

_headers 파일로
CSP 등 설정 가능

03 Pages 에서 할 수 있는 것

meta 태그 CSP(일부 제한)
외부 스크립트 최소화

어느 곳이든 HTTPS 를 켜고, API 키·개인정보를 코드에 넣지 않는 것이 기본입니다

PART 7

GitHub Pages 배포

내 컴퓨터의 코드를 GitHub 에 올리고, 누구나 열 수 있는 주소로 공개합니다

Git 은 저장 기록 장치

커밋

01

그 순간의 상태를
사진처럼 찍어 둔다

브랜치

02

원본을 건드리지 않는
평행 작업 공간

원격 저장소

03

GitHub 에 올린
내 저장소의 사본

비유 — 게임의 세이브 포인트. 자주 저장해 두면 망쳐도 그 지점으로 돌아갈 수 있습니다

자주 쓰는 Git 명령

명령	하는 일
git status	바뀐 파일·올릴 준비 상태 보기
git add .	바뀐 파일을 모두 올릴 준비
git commit -m "메시지"	지금 상태를 기록으로 저장
git log --oneline	저장 기록 훑어보기
git push	GitHub 로 올리기
git pull	GitHub 의 최신 내용 받기

AI 에이전트에게 커밋을 맡겨도, 무엇이 올라가는지는 git status 로 직접 확인합니다

GitHub 에 빈 저장소 만들기

- 1 오른쪽 위 + → New repository**
GitHub 에 로그인한 상태에서
- 2 저장소 이름 입력**
예 my-portfolio — 주소의 일부가 된다
- 3 Public 선택**
무료 계정은 공개 저장소에서 Pages 를 쓸 수 있다
- 4 Create repository**
README 추가 없이 빈 저장소로 만든다

특별한 이름

[아이디].github.io
= 바로 사이트 주소

커밋하고 GitHub 로 올리기

```
1 git add .
2 git commit -m "포트폴리오 첫 버전"
3
4 git remote add origin https://github.com/[아이디]/[저장소].git
5 git branch -M main
6 git push -u origin main
```

주소는 저장소 화면의 HTTPS 주소를 복사합니다 — 첫 푸시에만 -u 를 붙입니다

GitHub Pages 설정

1

저장소 Settings 탭

저장소 위쪽 메뉴

2

왼쪽 메뉴 Pages

Code and automation 묶음 안

3

Deploy from a branch

Branch: main · 폴더: / (root) → Save

4

주소 확인

몇 분 뒤 위쪽에 주소가 뜬다 — 이후 main 에 push 하면 자동 반영

내 사이트 주소

[아이디].github.io
/[저장소]/

에이전트에게 배포 전 점검 맡기기

구조 — 점검 목록 + 보고 형식 + 승인

“GitHub Pages 에 올리기 전에 이 폴더를 점검해 줘. ① 파일 이름 대소문자와 HTML 안의 경로가 정확히 같은지 ② 절대 경로(/로 시작) 대신 상대 경로를 쓰는지 ③ .env·개인정보가 올라갈 파일에 없는지 ④ index.html 이 맨 위 폴더에 있는지. 문제 목록만 먼저 보여 주고, 고치는 것은 내가 승인한 뒤에 해 줘.

이유

Pages 서버는 대소문자를 구분한다 — 내 PC에선 되던 것이 깨진다

커스텀 도메인 연결

도메인이 있다면 [아이디].github.io 대신 내 주소로 열 수 있습니다

DNS 설정

도메인 업체에서
CNAME → [아이디].github.io

Pages 입력

Settings → Pages
Custom domain 저장

연결 확인

DNS 확인이 끝날 때까지
기다린다(시간 소요)

HTTPS

Enforce HTTPS
체크

PART 8

문제 해결과 최종 점검

배포 뒤에 자주 만나는 문제와, 공개 전에 마지막으로 볼 것

자주 만나는 문제와 해결

404	Pages 설정(main·/root) 확인, index.html 위치·이름 확인
그림 깨짐	Logo.png ≠ logo.png — git mv 로 소문자 이름 통일
반영 안 됨	배포에 몇 분 걸린다 — 기다린 뒤 강력 새로고침
푸시 거절	git pull 로 원격 변경을 먼저 받고 다시 push
CORS 오류	브라우저가 다른 출처 요청을 막은 것 — 다음 장

외부 API 가 막힐 때(CORS)

왜 막히나

01

API 서버가 내 사이트 주소를
허용하지 않았기 때문

바른 해결

02

자기 백엔드나 서버리스
함수를 거쳐 호출한다

개발 중에만

03

Vite 같은 개발 서버의
프록시 설정(배포엔 안 됨)

남이 운영하는 공개 CORS 프록시는 쓰지 않습니다 — 내 요청과 데이터가 남의 서버를 지나갑니다

공개 전 최종 점검

● 점검

성능	이미지 압축·형식 확인, Lighthouse 지적 항목 정리
검색	title·description·OG 태그가 내 내용인지
접근성	alt·키보드 이동·명도 대비
보안	HTTPS 켜짐, 카·개인정보가 코드에 없음
품질	콘솔 오류 0, 깨진 링크 없음, 휴대폰에서 직접 열어 봄

배포 주소를 휴대폰으로 열어 보는 것이 마지막 확인

A07 마무리

바이트코딩 포트폴리오 배포 마무리

AI 는 손을 빠르게 해 줄 뿐 — 무엇을 만들지, 잘 됐는지는 내가 판단합니다

✓ 터미널 에이전트로 폴더 안에서 함께 작업

✓ 역할·목표·제약·맥락으로 요청

✓ 작게 만들고 브라우저에서 바로 확인

✓ GitHub Pages 로 공개하고 점검

오늘의 산출물

[아이디].github.io 의 내 포트폴리오

CREDITS · CONTACT

출처 · 문의

이애본 (Aebon Lee, Ph.D) — 드림아이티비즈 대표 · 한신대학교 AI·SW대학 겸임교수

useful-ai.dreamitbiz.com — 강의안·영상 강의 라이브러리

aebon@dreamitbiz.com · 카카오톡 ID aebon

© 2026 이애본 · 발행 홍문원(洪文院) · CC BY-NC-ND 4.0